

부품제작 · AI 활용 기술서

이윤수

August 7, 2026

부품제작 AI 활용 기술서

NAN 2026 Game x AI Hackathon 사전 과제

개인 참가자: 이윤수

작성일: 2026년 8월 7일

소스 코드: <https://github.com/yunsulee2/bupum-jejak>

플레이: <https://yunsulee2.github.io/bupum-jejak/>

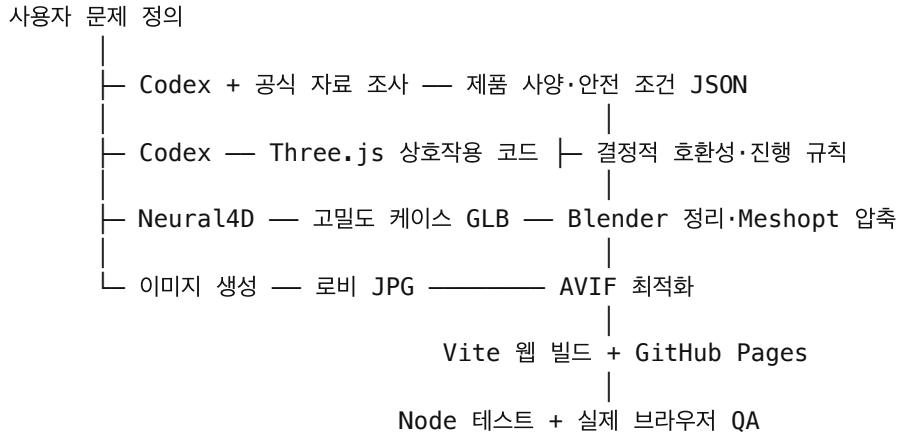
1. AI 활용 요약

부품제작은 AI를 기획·조사·코드 제작·3D 자산 생성·시각 제작·반복 검증에 사용했습니다. 게임 실행 중에는 외부 AI API를 호출하지 않습니다. 플레이 결과는 공개된 제품 규격을 구조화한 JSON과 명시적인 호환성 규칙으로 결정됩니다. 이 구분 덕분에 심사 환경에서도 API 키, 네트워크 응답, 확률적 답변에 의존하지 않고 같은 입력에 같은 결과를 냅니다.

AI 도구	사용 영역	결과물	사람의 검토
OpenAI Codex	요구사항 구체화, 웹 조사, Three.js·Vite 구현, Blender Python 스크립트, 테스트·브라우저 QA	소스 코드, 데이터 구조, 테스트, 문서	실습 주제·제품 범위·상호작용 방향 결정, 결과 확인
Neural4D 2.5	강화유리 ATX 케이스의 고밀도 3D 초안과 PBR 표면 생성	약 100만 면·4K PBR GLB 원본	실제 비례 조정, 메시 최적화, 조립 노드와 충돌영역 재구성
OpenAI 이미지 생성	여러 생활 조립 작업을 한 공간으로 묶는 메인 배경 제작	16:9 로비 배경 JPG	제품 범위 확인, UI 가독성에 맞춘 크롭·명암·AVIF 최적화

AI는 초안을 빠르게 만들었지만 제품 호환성, 안전 문구, 최종 동작 규칙은 코드와 테스트에서 다시 명시했습니다. AI의 자연어 답변을 정답 판정에 직접 사용하지 않았습니다.

2. 전체 구조



런타임 구성

- `src/main.js` : 첫 화면과 실습별 지연 로딩
- `src/app.js` : 공통 Three.js 장면, PC 구매·14단계 조립, 카메라와 완성 연출
- `src/fluorescent-module.js` : 형광램프 규격·안정기 호환, 90° 회전, 점등·배출
- `src/shower-filter-module.js` : 본체 세대·리필 호환, 360° 캡 조작, O링·누수 검사
- `src/drawer-module.js` : 판재 방향, 캠락, 좌우 레일, 직각, 전도 방지, 동작 시험
- `public/data/*.json` : 제품 사양, 안전 중단 조건, 단계, 출처
- `assets/pc-lab-source.blend` : 편집 가능한 Blender 원본
- `assets/neural4d/atx-case-source.glb` : Neural4D 고밀도 원본
- `public/models/pc-lab*.glb` : 웹에서 사용하는 최적화 모델

3. AI를 사용한 제작 과정

3.1 문제를 게임 규칙으로 변환

사용자는 “설명서대로 클릭하는 게임”이 아니라 제품마다 다른 실무 문제를 배우고, 실제 부품을 직접 끌어 설치하는 게임을 요구했습니다. Codex는 이를 다음 공통 상태로 분해했습니다.

1. 기존 제품이나 공간 조사
2. 후보 제품 비교
3. 안전 잠금
4. 직접 조작
5. 작동 검증
6. 결과와 실수 원인 정리

그 뒤 제품별로 다른 규칙을 분리했습니다. PC는 가격·부품 장착·첫 부팅, 형광등은 램프 코드·핀·전력·안정기, 샤워기는 본체 세대·리필 계열·O링, 서랍장은 판재 방향·레일 좌우·직각·전도 방지를 사용합니다.

3.2 제품·안전 정보 조사

Codex의 웹 조사 기능으로 제조사 사양, 조립 설명서, 공공기관 안전 지침을 확인했습니다. 조사 결과를 그대로 복사하지 않고 판정에 필요한 필드만 JSON으로 구조화했습니다. 예를 들어 형광등 호환은 다음 조건을 모두 비교합니다.

기술 방식 + 관 형태 + 베이스 + 소비전력 + 길이 허용 오차
+ 색온도 + 안정기 방식

샤워기 필터는 브랜드만 맞는 것으로 처리하지 않고 `브랜드 + 제품군 + 세대 + 장착 위치 + 형태`가 일치해야 합니다. 출처는 각 데이터 파일의 `source` 또는 `sources` 항목에 보존했습니다.

3.3 코드 생성과 검증

Codex는 Three.js 기반 직접 조작, 방향 힌트, 거리 판정, 카메라 제어, 팬·RGB·점등·누수·서랍 움직임을 구현했습니다. 구현 후에는 정적 문자열 검사가 아니라 데이터 개수, 호환 제품 집합, 필수 노드, 모델 용량, 모바일 자산 선택까지 확인하는 Node 테스트를 추가했습니다.

현재 자동 검증은 21개이며 주요 항목은 다음과 같습니다.

- PC 가구·부품 14단계와 28개 구매 선택지
- 두 개의 정답 형광램프와 유사 오답 제품 구분
- 두 개의 정답 샤워기 리필과 다른 세대·타사 제품 구분
- 서랍장 8단계, 좌우 레일, 직각, 전도 방지
- Neural4D·Blender 원본과 Meshopt GLB 필수 노드
- 데스크톱·모바일 렌더링 자산과 성능 예산
- 모듈별 지연 로딩과 실패 후 재시도 상태

3.4 NEURAL4D와 BLENDER 결합

Neural4D가 만든 케이스는 표면 품질은 높았지만, 그대로는 약 100만 면·27 MB라 웹 상호작용용으로 무거웠고 조립 가능한 부품 ID도 없었습니다. 다음 후처리를 적용했습니다.

1. 고밀도 GLB를 Blender 장면에 가져옵니다.
2. 실제 ATX 케이스 비례와 조립 작업대 좌표에 맞춥니다.
3. 강화유리·외부 셸은 Neural4D 메시를 사용합니다.
4. CPU, 메인보드, GPU, 케이블과 팬은 결정적인 이름을 가진 프로젝트 전용 메시로 유지합니다.
5. 데스크톱 약 180,000면·6.3 MB, 모바일 약 3.2 MB로 줄입니다.
6. Meshopt 압축 뒤에도 `part_case`, `part_motherboard`, `part_gpu`, `part_cables` 노드가 남는지 테스트합니다.

이 방식은 AI 자산의 시각 품질과 게임 규칙에 필요한 안정적인 노드 구조를 분리합니다.

4. 주요 프롬프트와 지시 기록

아래는 실제 개발 대화에서 사용한 주요 지시를 핵심 문장 중심으로 정리한 것입니다. 비밀키와 개인 정보는 제외했습니다.

제품 조립 게임의 시작

“실제 데스크톱 PC에 들어가는 부품 정보를 조사하고 참고 이미지까지 반영해 구현해줘. 보여지는 모든 것이 부품이고 퀄리티는 최상이어야 해.”

결과: 실제 ATX 부품 구성, 제품별 선택지, Blender 생성기, Three.js 조립 장면을 만들었습니다.

직접 조작과 안내 방식

“마우스로 클릭해 끌어서 사용자가 해당 위치에 직접 설치하고, 어디에 설치해야 하는지 알려주는 게임이었으면 좋겠어. 한 부품도 여러 종류가 있었으면 좋겠어.”

결과: 부품 선택, 목표 홀로그램, 화면 거리 판정, 방향 힌트, 실패 복귀, 제품별 변형을 만들었습니다.

구매 결과가 실제 모델에 반영되는 흐름

“모니터부터 데스크톱 PC, 의자 등을 먼저 구매하고 조립한 뒤, 완성되면 방에 실제로 적용된 모습을 보여줘. 구매한 것에 따라 텍스트뿐 아니라 부품도 바뀌어야 해.”

결과: 28개 제품 선택, 실시간 총액, 구매 SKU별 색·형태·팬 수, 최종 게이밍 룸을 연결했습니다.

3D 품질과 NEURAL4D

“실제 부품 같지 않아 아쉽다. 3D 에셋 품질을 높이고, 강화유리 케이스와 계속 회전하는 팬을 넣어줘.”

Neural4D 작업 핵심 지시를 재구성하면 다음과 같습니다.

“검은색 강화유리 ATX 미들타워 케이스. 측면과 전면 유리, 실제 금속 패널 두께, 내부 프레임과 통풍구, PBR 금속·유리 재질, 게임용 제품 촬영 수준의 비례와 디테일.”

결과: 고밀도 PBR 케이스 셀을 생성하고 Blender에서 조립 가능한 구조로 결합했습니다.

제품마다 다른 문제 해결

“형광등은 종류를 먼저 파악하고 맞는 제품을 산 뒤 교체해야 해. 컴퓨터 조립과 다른 사항을 조사해서 개발해줘.”

“샤워기 필터는 본체를 살피 필요한 필터를 고르고 누수 없이 교체하는 흐름으로 추가해줘.”

“서랍처럼 더 대중적으로 조립할 수 있는 것을 구상해서 개발해줘.”

결과: 동일한 드래그 게임을 복제하지 않고 조사 단서, 호환 판정, 안전 잠금, 제품별 전용 제스처와 검증 조건을 각각 구현했습니다.

5. 사람과 AI의 역할 구분

단계	사람	AI
문제 선택	집에서 실제로 겪는 조립·교체 문제와 우선순위 결정	문제를 게임 상태와 화면 흐름으로 분해
제품 범위	PC, 형광등, 샤워기 필터, 서랍장으로 확장 결정	제품 사양과 절차 조사, 데이터 초안 작성
시각 방향	실제 부품처럼 보이고 직접 만지는 느낌 요구	3D 생성·후처리 스크립트·조명·재질 구현
품질 판단	공중 조립, 케이블 위치, 작은 글자 등 문제 제기	원인 분석, 코드 수정, 회귀 테스트 추가
최종 책임	제출 범위와 외부 자료 사용 여부 승인	문서화, 자동 검사, 재현 가능한 빌드 제공

6. 보안과 재현성

- 저장소와 프로덕션 번들에 Neural4D 또는 OpenAI API 키를 포함하지 않았습니다.
- 게임 실행 중 외부 AI API를 호출하지 않습니다.
- 제품 데이터와 판정 규칙은 저장소에 있어 심사자가 확인할 수 있습니다.
- Git 커밋 기록을 보존했습니다.
- `npm ci`, `npm test`, `npm run build` 로 같은 결과를 재현할 수 있습니다.
- GitHub Pages 배포는 GitHub Actions의 최소 권한과 OIDC를 사용하며 별도 배포 비밀키가 필요하지 않습니다.

7. 외부 자산·오픈소스·라이선스

코드와 도구

항목	버전·범위	라이선스·권리 상태
Three.js	0.179.1	MIT License, three.js authors
Vite	7.3.6	MIT License, VoidZero Inc. 및 기여자
Blender	3D 편집·Python 생성 파이프라인	Blender는 GPL, Blender로 만든 프로젝트 출력물에는 GPL이 자동 적용되지 않음
OpenAI Codex· 이미지 출력	코드·문서·로비 이 미지 제작 보조	OpenAI 이용약관상 법이 허용하는 범위에서 입력 권리는 유지되 고 출력은 사용자에게 귀속. Codex 출력에 포함될 수 있는 제3자 라이선스는 별도 검토
Neural4D 2.5 출 력	ATX 케이스 GLB	Neural4D는 입력·출력 소유권을 주장하지 않음. 상업적 이용 범위 는 생성 당시 계정 플랜과 최신 약관에 따름

Three.js와 Vite의 라이선스 전문은 설치 패키지의 `LICENSE` 파일에 포함되어 있습니다. 관련 공식 링크와 외부 자료 전체 목록은 저장소의 `submission/ASSET_SOURCES.md`에 정리했습니다.

프로젝트 제작 자산

- `assets/pc-lab-source.blend` : 프로젝트에서 만든 편집 가능한 Blender 장면
- `tools/build_pc_scene.py` : 프로젝트에서 만든 메시·재질 생성 스크립트
- `assets/neural4d/atx-case-source.glb` : Neural4D 출력 원본
- `public/models/pc-lab*.glb` : 위 원본과 프로젝트 메시지를 결합한 최적화 출력
- `public/images/home-diy-hero-v1.*` : OpenAI 이미지 생성 결과를 로비용으로 최적화
- `qa/*.png` : 프로젝트를 실제 실행해 직접 캡처한 QA 스크린샷
- 효과음: 외부 음원 파일 없이 Web Audio API로 실행 중 합성

제품 사진과 상표

구매 화면의 제품 사진은 제조사·공식 판매처·가격 비교 페이지의 공개 제품 이미지를 제품 식별과 비교를 위해 로컬에 저장한 것입니다. 각 항목의 원문 URL은 `public/data/desktop-atx.json`, `store-catalog.json`, `shower-filter-lab.json`에 연결했습니다.

이 사진들은 오픈 라이선스로 확인되지 않았으며 저작권과 상표권은 각 권리자에게 있습니다. 별도 재사용 허락은 확인되지 않았습니다. 본 사전 과제에서는 비공식 교육용 프로토타입의 제품 식별 목적으로만 사용하고, 원본 사진을 별도 배포하거나 공식 제휴·보증으로 표현하지 않습니다. 공개·상용 전환 시에는 권리자 허락을 받거나 직접 제작·정식 라이선스 자산으로 교체해야 합니다.

8. 한계와 다음 단계

현재 AI는 제작 파이프라인에 사용되며 플레이 도중 답변을 생성하지 않습니다. 따라서 오프라인에서도 안정적이고 판정 근거를 설명할 수 있지만, 사용자의 반복 실수 유형을 장기적으로 학습하거나 자유로운 질문에 답하지는 못합니다.

다음 버전에서는 사용자의 오답 순서와 거리·방향 기록을 로컬에서 분석해 “정답 위치” 대신 “왜 이 선택이 틀렸는지”를 개인화하는 조립 코치를 추가할 계획입니다. 안전이 관련된 답변은 생성 모델 단독 판정이 아니라 검증된 규칙과 출처를 먼저 통과시키는 구조를 유지합니다.